

TP1 - R4.Cyber.9

Chiffrement symétrique AES en mode CBC

CBC (Cipher Block Chaining) est un mode de fonctionnement de l'AES dont la force réside dans sa capacité à obfusquer de manière fiable les motifs du texte en clair. Pour ce faire, un vecteur d'initialisation est utilisé dans le premier bloc puis, avant d'être chiffré, une opération de "ou exclusif" ("XOR") est réalisée entre chaque bloc suivant de texte en clair et le bloc précédent chiffré. Ainsi, chaque bloc de texte chiffré dépend de la chaîne de blocs de texte qui le précède.

- Créez un fichier `test.txt` contenant le mot test en une ligne de commande Linux.
 - *Aide/rappel :* `echo test > test.txt`
 - `cat test.txt` pour voir le contenu rapidement
- Chiffrez ce fichier en un fichier `test.enc` avec l'algorithme AES mode CBC avec openssl (voir mémento).
 - Retenez bien votre mot de passe. Utilisez progtr00 par exemple (pas un vrai perso!).
- Ouvrez le fichier chiffré pour voir ce qu'il contient.
 - *Aide/rappel :* commande `cat`
- Envoyez ce fichier à un voisin/binôme.
- Comment peut-il le déchiffrer ? Exécutez la commande de déchiffrement de ce fichier.
 - *Cf cours :* Oui il doit connaître aussi le mot de passe protégeant l'usage de la clé. Avec ce chiffrement une seule clé pour chiffrer et déchiffrer.

On commence par créer le fichier test.txt :

```
raphael_laurent@j009-10:~$ echo test > test.txt
raphael_laurent@j009-10:~$ cat test.txt
test
```

On chiffre le fichier avec la commande `openssl enc -aes-256-cbc -salt -in test.txt -out test.enc`

```
raphael_laurent@j009-10:~$ openssl enc -aes-256-cbc -salt -in test.txt -
out test.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

On tente d'ouvrir le fichier encrypter et on obtiens :

```
raphael_laurent@j009-10:~$ cat test.enc
Salted__
    q$    KE    Zh    raphael_laurent@j009-10:~$
```

On envoie le fichier a notre binome via notion :

test.enc

Pour déchiffrer le fichier il peut utiliser la commande `openssl enc -d -aes-256-cbc -in test.enc -out test.txt` et le mot de passe est progtr00 :

```
cyril_godart@j009-09:~/Téléchargements$ openssl enc -d -aes-256-cbc -in test.enc  
-out test.txt  
enter AES-256-CBC decryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.  
cyril_godart@j009-09:~/Téléchargements$ cat test.txt  
test  
cyril_godart@j009-09:~/Téléchargements$
```

Fonctions de hachage

Une fonction de hachage (hash function)

convertit toute donnée numérique en une chaîne de sortie comportant un nombre fixe de caractères, ce hachage est une action unidirectionnel de

conversion des données (appelées `message`) en données de sortie (appelées `hachage`, `hash`, `haché`, ou encore `empreinte`).

Le `hachage` permet par exemple :

- de garantir l'authenticité des données
- de vérifier l'intégrité des données, en permettant de détecter les modifications apportées aux données d'origine.
- de stocker des mots de passe (on ne stocke jamais des mots de passe en clair, on stocke leurs empreintes/hashs)
- de créer des bases d'empreintes de virus pour les antivirus
- d'éviter le transfert de gros fichiers si déjà reçus

Les principaux algorithmes sont :

- `MD5 (Message-Digest Algorithm 5) (cassé!)` : une fonction de hachage très populaire, mais qui a été cassée et n'est plus considérée comme sûre pour de nombreuses applications.
- `SHA (Secure Hash Algorithm) (SHA-1 cassé!)` : une famille de fonctions de hachage développée par l'agence de sécurité nationale des États-Unis (NSA), qui comprend les versions SHA-1, SHA-2 et SHA-3.
- `BLAKE2` : une fonction de hachage rapide et sécurisée, développée pour remplacer MD5 et SHA-1.
- `Argon2` : une fonction de hachage qui a remporté le concours international de sécurité de hachage en 2015 et qui est considérée comme l'une des meilleures fonctions de hachage actuelles.
- Créez un fichier `test.sha256` contenant le hash sha256 du contenu du fichier `test.txt`, et un fichier `test.sha1` contenant le hash sha1 du contenu du fichier `test.txt`.
- Retrouvez par le calcul via des commandes bash les longueurs des hashs sha1 et sha256.
 - *Aide :*
 - `hash1=4e1243bd22c66e76c2ba9eddc1f91394e57f9f83`
 - `echo ${#hash1}`

- Tentez de retrouver les hashes inverses par exemple via ce site : <https://md5hashing.net/hash>

On commence par créer un fichier test.sha256 avec la commande `openssl dgst -sha256 -out test.sha256 test.txt`

```
raphael_laurent@j009-10:~$ openssl dgst -sha256 -out test.sha256 test.txt
raphael_laurent@j009-10:~$ cat test.sha256
SHA2-256(test.txt)= f2ca1bb6c7e907d06dafa4687e579fce76b37e4e93b7605022da52e6ccc26fd2
raphael_laurent@j009-10:~$
```

Puis on crée le fichier test.sha1 avec la commande `openssl dgst -sha1 -out test.sha1 test.txt` :

```
raphael_laurent@j009-10:~$ openssl dgst -sha1 -out test.sha1 test.txt
raphael_laurent@j009-10:~$ cat test.sha1
SHA1(test.txt)= 4e1243bd22c66e76c2ba9eddc1f91394e57f9f83
```

On essaie ensuite de retrouver par calcul la longueur des hash :

```
raphael_laurent@j009-10:~$ cat test.sha1
SHA1(test.txt)= 4e1243bd22c66e76c2ba9eddc1f91394e57f9f83
raphael_laurent@j009-10:~$ hash1=4e1243bd22c66e76c2ba9eddc1f91394e57f9f83
raphael_laurent@j009-10:~$ echo ${#hash1}
40
raphael_laurent@j009-10:~$ cat test.sha256
SHA2-256(test.txt)= f2ca1bb6c7e907d06dafa4687e579fce76b37e4e93b7605022da52e6ccc26fd2
raphael_laurent@j009-10:~$ hash2=f2ca1bb6c7e907d06dafa4687e579fce76b37e4e93b7605022da52e6ccc26fd2
raphael_laurent@j009-10:~$ echo ${#hash2}
64
```

On tente de retrouver les hashes inverses via <https://md5hashing.net/hash> :

Sha1 hash calculated hash digest 4e1243bd22c66e76c2ba9eddc1f91394e57f9f83  Copy Hash	Sha1 value Reversed hash value test  Copy Value Blame this record
---	---

Sha256 hash calculated hash digest f2ca1bb6c7e907d06dafa4687e579fce76b37e4e93b76 05022da52e6ccc26fd2 Copy Hash	Sha256 value Reversed hash value test Copy Value Blame this record
---	---

Codage base64

Le codage base64

utilise 64 caractères classiques (majuscules + minuscules + chiffres + caractère "+" + caractère "/"), soit [A-Za-z0-9+/], ainsi qu'un 65e caractère "=" utilisé comme complément pour toujours avoir à coder des groupes de 24 bits = 3 octets = 4×6 bits. Dans ce codage, un caractère code 6 bits, aussi 3 octets quelconques sont donc remplacés par 4 caractères.

Un inconvénient est que ce codage augmente la taille des données.

L'avantage principal est que toute donnée (image/musique/vidéo/exécutible/...) est représentable en 64 ou 65 caractères affichables (textuels).

Attention c'est un codage pas un chiffrement.

- Codez avec openssl le contenu du fichier `test.txt` en base64 dans un fichier `test.base64`, puis décoder ce fichier dans un fichier `test_decode.txt`.
 - *Aide : si votre fichier contient bien 5 caractères : test + retour à ligne, le codage obtenu doit être `dGVzdAo=`*
- Vérifiez ce résultat avec des commandes bash.
 - `echo "test" | base64`
 - `echo -n "test" | base64`
- Ce codage permet d'afficher des données non textuelles, vérifiez le avec une commande bash en décodant ce code `CdZ4MLMPgYtAE9gQ80gMtg==`, utilisez aussi la commande `xxd` permettant d'afficher la valeur hexadécimale ou binaire de n'importe quelle donnée.

On encode le fichier test.txt en base 64 avec la commande `openssl base64 -in test.txt -out test.base64` et on le decode avec la commande `openssl base64 -d -`

`in test.base64 -out test_decode.txt`, on obtiens :

```
raphael_laurent@j009-10:~$ openssl base64 -in test.txt -out test.base64
raphael_laurent@j009-10:~$ cat test.base64
dGVzdAo=
raphael_laurent@j009-10:~$ openssl base64 -d -in test.base64 -out test_decode.txt
raphael_laurent@j009-10:~$ cat test_decode.txt
test
```

On vérifie que ça correspond bien a test avec les commandes `echo "test" | base64` et `echo -n "test" | base64`

```
raphael_laurent@j009-10:~$ echo "test" | base64
dGVzdAo=
raphael_laurent@j009-10:~$ echo -n "test" | base64
dGVzdA==
```

On essaie de décoder le code `CdZ4MLMPgYtAE9gQ80gMtg==` avec la commande `echo "CdZ4MLMPgYtAE9gQ80gMtg==" | base64 -d | xxd`

```
raphael_laurent@j009-10:~$ echo "CdZ4MLMPgYtAE9gQ80gMtg==" | base64 -d | xxd
00000000: 09d6 7830 b30f 818b 4013 d810 f348 0cb6  ..x0....@....H..
```

Chiffrement asymétrique RSA

- Créez un nouveau fichier `texte.txt` contenant un texte que vous souhaitez envoyer à un voisin.
- Avec openssl et un chiffrement `RSA` (voir mémento), effectuez les opérations permettant de transmettre ce fichier chiffré à votre voisin qui pourra le déchiffrer.
 - *Aide (cf cours et mémento) vous devez utiliser la clé publique du voisin pour lui envoyer des messages que lui seul (elle seule) pourra déchiffrer en utilisant la clé privée associée à cette clé publique.*

Notre voisin génère sa clé privé et publique avec les commandes :

```
openssl genrsa -out private.key 2048
```

```
openssl rsa -in private.key -out public.key -pubout
```

On crée le fichier texte.txt :

```
raphael_laurent@j009-10:~$ echo "coucou Cyril" > texte.txt
```

On demande la clé publique de notre voisin :

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAtHzMsKiyP6v0KcX
UHfyY
FD9Zk3R87Rg8fFYeXGYB4a0NDDfAtsJ9IYui95BX3pVFg5Zf8X2H9BS6cu8
BK0Xu
UFHqwA0d7tVrISlHY4yKNo+Y+Am9/wmICjafGbwLF2F3AfjYYFQ1Rr71L7W
P9mBt
+2EnQsfh6tKjQYq85tL8dMD3v9kyC0h6R4GTOLlgNCLvkBq0j5+wfaUViJw
7rgMo
kMT6EC+eX9F40p3raK09hYgqKbxUNFRQ2VAio300WXheYozJJLeMQoCq6Kz
CN2va
YHAW00YTdV/mhHzVocZ28lIVJHkwJKDvdyqvK09UtLYsWzqoes6T0Jeb1Tz
0+BQU
WwIDAQAB
-----END PUBLIC KEY-----
```

On utilise la clé public de notre voisin et on encrypt le fichier avec sa clé publique :

```
raphael_laurent@j009-10:~$ echo "-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAtHzMsKiyP6v0KcXUHfyY
FD9Zk3R87Rg8fFYeXGYB4a0NDDfAtsJ9IYui95BX3pVFg5Zf8X2H9BS6cu8BK0Xu
UFHqwA0d7tVrISlHY4yKNo+Y+Am9/wmICjafGbwLF2F3AfjYYFQ1Rr71L7WP9mBt
+2EnQsfh6tKjQYq85tL8dMD3v9kyC0h6R4GTOLlgNCLvkBq0j5+wfaUViJw7rgMo
kMT6EC+eX9F40p3raK09hYgqKbxUNFRQ2VAio300WXheYozJJLeMQoCq6KzCN2va
YHAW00YTdV/mhHzVocZ28lIVJHkwJKDvdyqvK09UtLYsWzqoes6T0Jeb1Tz0+BQU
WwIDAQAB
-----END PUBLIC KEY-----" > cyril_public.key
raphael_laurent@j009-10:~$ openssl rsautl -encrypt -inkey cyril_public.key -pubin
-in texte.txt -out texte.enc
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
```

On lui envoie le fichier encrypter :

texte.enc

Il peut le décrypter avec la commande `openssl rsautl -decrypt -inkey private.key -in texte.enc -out raph_texte.txt` :

```
cyril_godart@j009-09:~/Téléchargements$ openssl rsautl -decrypt -inkey private.k
ey -in texte.enc -out raph_texte.txt
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
cyril_godart@j009-09:~/Téléchargements$ cat raph_texte.txt
coucou Cyril
```

Signature

Rappel cours : classiquement la signature d'une donnée peut se faire en chiffrant avec sa clé privée le hash (sha256 par exemple) de ce document (chiffrer un hash est plus rapide et plus léger que chiffrer toute une donnée, en particulier un gros fichier). La vérification se fera alors via la clé publique associée.

- Créez un nouveau fichier `document.txt` contenant un texte quelconque par exemple "document".
- Signez avec openssl votre document et envoyez le à un voisin qui doit pouvoir vérifier votre signature.
 - *Aide : voir mémento*

On crée un document.txt et on le signe avec les commandes `echo "document ultra sensible pour cyril" > document.txt` et `openssl dgst -sign private.key -sha256 -out document.sign document.txt` :

```
raphael_laurent@j009-10:~$ echo "document ultra sensible pour cyril" > document.txt
raphael_laurent@j009-10:~$ openssl dgst -sign private.key -sha256 -out document.sign
document.txt
```

On transfère le document à notre voisin :

document.sign

document.txt

Notre voisin le déchiffre avec la commande `openssl dgst -verify raph_public.key -sha256 -signature document.sign document.txt` :

```
cyril_godart@j009-09:~/Téléchargements$ openssl dgst -verify raph_public.key -sha256 -signature document.sign document.txt
Verified OK
```

Certificats SSL/TLS

Les `certificats SSL/TLS`

sont des certificats numériques utilisés pour établir des connexions sécurisées entre appareils en réseau dans Internet, par exemple un navigateur (client) et un serveur Web. Ces certificats utilisent les protocoles Secure Sockets Layer (SSL) et Transport Layer Security (TLS) pour chiffrer les données et garantir qu'elles ne sont accessibles qu'aux destinataires prévus, sachant que TLS est plus récent et sûr qu'SSL.

Les `certificats SSL/TLS` sont émis par des `autorités de certification (CA - Certificate Authority)`, qui sont des `tiers de confiance` qui vérifient l'identité des organisations ou des personnes demandant les certificats.

Pour ce faire, l'autorité de certification (CA) signe un certificat avec sa clé privée, tous les clients peuvent alors vérifier la signature avec la clé publique connu de tous. Par exemple tous les navigateurs connaissent les clés publiques des CA publiques de confiance. Parmi les CA les plus connus, on trouve CAcert.org, DigiCert, Verisign et Let's Encrypt.

- Récupérez le certificat du site web du département `https://rt-bethune.univ-artois.fr`.
 - `echo | openssl s_client -connect rt-bethune.univ-artois.fr:443 | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > rtbethune.crt`
 - Attention à l'IUT, besoin de préciser le proxy, on peut le faire avec l'option `-proxy` de openssl : `proxy cache-etu.univ-artois.fr:3128`
- Puis affichez le
 - `openssl x509 -in rtbethune.crt -text`

- Observez l'autorité de certification et récupérez également son certificat, affichez le et continuez ainsi jusqu'à remonter toute la chaîne de certification.
 - Remarquez qu'il existe une chaîne de certificats, c'est-à-dire une collection hiérarchique de certificats. Il est en effet possible de construire à partir d'un CA racine une chaîne de CA étant donné que toutes les parties approuvent le certificat racine.

On récupère le certificat du site de l'IUT avec la commande `echo | openssl s_client -connect rt-bethune.univ-artois.fr:443 -proxy cache-etu.univ-artois.fr:3128 | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > rtbethune.crt :`

```
echo | openssl s_client -connect rt-bethune.univ-artois.fr:443 -proxy cache-etu.univ-artois.fr:3128 | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > rtbethune.crt
```

```
raphael_laurent@j009-10:~/Téléchargements$ echo | openssl s_client -connect rt-bethune.univ-ar  
tois.fr:443 -proxy cache-etu.univ-artois.fr:3128 | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIF  
ICATE-/p' > rtbethune.crt  
  
depth=2 C = GR, O = Hellenic Academic and Research Institutions CA, CN = HARICA TLS ECC Root C  
A 2021  
verify return:1  
depth=1 C = GR, O = Hellenic Academic and Research Institutions CA, CN = GEANT TLS ECC 1  
verify return:1  
depth=0 C = FR, ST = Hauts-de-France, L = Arras, O = UNIVERSIT\C3\89 D ARTOIS, CN = rt-bethune  
.univ-artois.fr  
verify return:1  
DONE
```

On lit le contenu du certificat avec la commande `openssl x509 -in rtbethune.crt -text` :

```
raphael_laurent@j009-10:~/Téléchargements$ openssl x509 -in  
rtbethune.crt -text
```

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

69:c9:80:94:7d:59:a0:0f:77:21:90:f6:78:cd:a4:d7

Signature Algorithm: ecdsa-with-SHA384

Issuer: C = GR, O = Hellenic Academic and Research

```

Institutions CA, CN = GEANT TLS ECC 1
  Validity
    Not Before: Sep 11 14:09:03 2025 GMT
    Not After : Sep 11 14:09:03 2026 GMT
  Subject: C = FR, ST = Hauts-de-France, L = Arras, O
= UNIVERSIT\C3\89 D ART0IS, CN = rt-bethune.univ-artois.fr
  Subject Public Key Info:
    Public Key Algorithm: id-ecPublicKey
    Public-Key: (256 bit)
    pub:
      04:f1:89:8b:84:47:43:e0:68:3a:c9:95:df:
a0:1f:
      61:a3:80:bc:fd:83:6c:42:ea:6a:2a:c8:e8:
2b:8f:
      fc:51:fc:68:3c:a5:41:93:5e:bd:69:00:60:
73:93:
      ff:92:c2:36:53:73:a2:e7:f5:34:89:88:a6:
6b:60:
      b9:70:06:24:3f
    ASN1 OID: prime256v1
    NIST CURVE: P-256
  X509v3 extensions:
    X509v3 Authority Key Identifier:
      E9:99:06:8D:17:1F:AB:FB:96:1A:5A:C8:5B:5E:5
D:5E:EC:DA:9C:8F
    Authority Information Access:
      CA Issuers - URI:http://crt.harica.gr/HARIC
A-GEANT-TLS-E1.cer
      OCSP - URI:http://ocsp-tls.harica.gr
    X509v3 Subject Alternative Name:
      DNS:rt-bethune.univ-artois.fr
    X509v3 Certificate Policies:
      Policy: 2.23.140.1.2.2
      Policy: 0.4.0.2042.1.7
      Policy: 1.3.6.1.4.1.26513.1.1.1.2
    X509v3 Extended Key Usage:
      TLS Web Client Authentication, TLS Web Serv
er Authentication

```

```

X509v3 CRL Distribution Points:
  Full Name:
    URI:http://crl.harica.gr/HARICA-GEANT-TLS
-E1.crl
X509v3 Subject Key Identifier:
  FF:06:12:53:4F:43:B2:7B:A0:EB:53:14:29:AD:F
F:A2:42:43:D7:56
X509v3 Key Usage: critical
  Digital Signature
CT Precertificate SCTs:
  Signed Certificate Timestamp:
    Version      : v1 (0x0)
    Log ID       : 94:4E:43:87:FA:EC:C1:EF:81:
F3:19:24:26:A8:18:65:
                                01:C7:D3:5F:38:02:01:3F:72:
67:7D:55:37:2E:19:D8
    Timestamp    : Sep 11 14:19:08.071 2025 GM
T
    Extensions: none
    Signature    : ecdsa-with-SHA256
                                30:45:02:20:73:43:90:6E:FD:
B4:39:98:DE:ED:99:5D:
                                44:72:46:0E:CE:AD:A7:3A:64:
3A:8E:E0:AA:38:A6:E3:
                                5F:06:CD:8F:02:21:00:94:87:
20:49:89:DD:4C:5F:09:
                                FE:14:47:F4:4D:01:96:E7:70:
2B:30:A7:18:45:BA:31:
                                63:12:30:24:6B:BE:D7
    Signed Certificate Timestamp:
    Version      : v1 (0x0)
    Log ID       : D7:6D:7D:10:D1:A7:F5:77:C2:
C7:E9:5F:D7:00:BF:F9:
                                82:C9:33:5A:65:E1:D0:B3:01:
73:17:C0:C8:C5:69:77
    Timestamp    : Sep 11 14:19:08.065 2025 GM
T
    Extensions: none

```

Signature : ecdsa-with-SHA256
30:46:02:21:00:A6:4C:36:BB:
C8:0D:30:7A:69:9B:75:
BC:34:69:30:6C:95:AD:36:8D:
E3:5F:03:61:25:E4:4A:
F5:A6:5C:91:C3:02:21:00:A9:
D1:BD:C7:86:45:B0:1A:
D7:4A:D8:5C:CF:2B:F9:9F:9F:
69:9F:85:28:8A:2B:E7:
43:FC:3F:9E:EB:B8:FD:51

Signed Certificate Timestamp:

Version : v1 (0x0)
Log ID : D8:09:55:3B:94:4F:7A:FF:C8:
16:19:6F:94:4F:85:AB:
B0:F8:FC:5E:87:55:26:0F:15:
D1:2E:72:BB:45:4B:14
Timestamp : Sep 11 14:19:08.024 2025 GM
T

Extensions: none

Signature : ecdsa-with-SHA256
30:46:02:21:00:C5:27:01:1E:
12:AB:53:CB:CC:A8:52:
A6:47:2D:31:B2:7F:B6:8D:59:
58:E6:53:7B:7F:58:61:
92:83:C0:6C:C0:02:21:00:E0:
D8:F9:38:F0:C0:11:CC:
3B:D0:3C:73:83:D0:AC:7D:33:
15:83:51:6F:84:E0:44:
66:A3:20:F9:CB:EC:B6:79

Signature Algorithm: ecdsa-with-SHA384

Signature Value:

30:65:02:30:29:1b:78:3c:2a:4e:da:80:54:0a:20:9a:9f:
97:
0f:1b:17:18:58:d7:39:73:6b:79:b0:40:a9:00:95:e0:17:
a0:
68:37:fc:14:a2:44:b2:01:59:72:34:bf:ee:bb:ec:ff:02:
31:
00:94:ed:28:3c:8a:ed:69:d0:81:2c:9d:bc:7b:8f:29:a3:

22:
11:d0:21:5e:e6:2f:5a:fe:8b:0a:38:8d:88:86:f7:a5:9d:
1f:
48:94:93:dc:93:33:0b:06:ea:c2:5d:68:00
-----BEGIN CERTIFICATE-----
MIIE9jCCBHygAwIBAgIQacmAlH1ZoA93IZD2eM2k1zAKBggqhkJOPQQDAzB
gMQsw
CQYDVQQGEwJHUjE3MDUGA1UECgwuSGVsbGVuaWMgQWNhZGVtaWMgYW5kIFJ
lc2Vh
cmNoIEluc3RpdHV0aW9ucyBDQTEYMBYGA1UEAwwPR0VBTLQgVExTIEVDQyA
xMB4X
DTI1MDkxMTE0MDkwM1oXDTI2MDkxMTE0MDkwM1owejELMAkGA1UEBhMCRLI
xGDAW
BgNVBAGMD0hhdXRzLWRLLUZyYW5jZTE0MAwGA1UEBwwFQXJyYXMxHTAbBgN
VBAoM
FFV0SVZFUlnNjVM0JIEQgQVJUT0ltMSIwIAAYDVQQDDBlldC1iZXRodW5lLnV
uaXYt
YXJ0b2l2LmZyMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE8YmLhEdD4Gg
6yZXf
oB9ho4C8/YNsQupqKsjokK4/8UfxoPKVBk169aQBgc5P/ksI2U30i5/U0iYi
ma2C5
cAYkP60CAvwggL4MB8GA1UdIwQYMBaAF0mZBo0XH6v7lhpayFteXV7s2py
PMG8G
CCsGAQUFBwEBBGMwYTA4BgggBgEFBQcwAoYsaHR0cDovL2NydC5oYXJpY2E
uZ3Iv
SEFSSUNBLUdFQU5ULVRMUy1FMS5jZXIwJQYIKwYBBQUHMAGGGWh0dHA6Ly9
vY3Nw
LXRscy5oYXJpY2EuZ3IwJAYDVRR0RBB0wG4IZcnQtYmV0aHVuZS51bml2LWF
ydG9p
cy5mcjAtBgNVHSAEJjAkMAgGBmeBDAECAjAIBgYEAi96AQcwDgYMKwYBBAG
BzxEB
AQECMB0GA1UdJQQWMBQGCCsGAQUFBwMCBgggBgEFBQcDATA9BgNVHR8ENjA
0MDKg
MKAuhixodHRwOi8vY3JsLmhhcmlljYS5nci9IQVJJQ0EtR0VBTLQ0tVExTLUU
xLmNy
bDAdBgNVHQ4EFgQU/wYSU09Dsnug61MUKa3/okJD11YwDgYDVRR0PAQH/BAQ
DAgeA
MIIBgAYKKwYBBAHWeQIEAgSCAXAEggFsAWoAdgCUTk0H+uzB74HzGSQmqBh

```

lAcfT
XzgCAT9yZ31VNy4Z2AAAAZk5JPenAAAEAwBHMEUCIHNDkG79tDmY3u2ZXUR
yRg70
rac6ZDq04Ko4puNfBs2PAiEAlIcgSYndTF8J/hRH9E0BludwKzCnGEW6MWM
SMCRr
vtcAdwDXbX0Q0af1d8LH6V/XAL/5gskzWmXh0LMBcxfAyMVpdwAAAZk5JPe
hAAAE
AwBIMEYCIQCMTDa7yA0wemmbdbw0aTBsla02jeNfA2El5Er1plyRwwIhAKn
RvceG
RbAa10rYXM8r+Z+faZ+FKIor50P8P57ruP1RAHcA2AlV05Rpev/IFhlvlE+
Fq7D4
/F6HVSYPFdEucrtFSxQAAAGZ0ST3eAAABAMASDBGAiEAxScBHhKrU8vMqFK
mRy0x
sn+2jVlY5lN7f1hhkoPAbMACIQDg2Pk48MARzDvQPHOD0Kx9MxWDUW+E4ER
moyD5
y+y2eTAKBggqhkJ0PQQDAwNoADBLAjApG3g8Kk7agFQKIJqflw8bFxbY1zl
za3mw
QKkAleAXoGg3/BSiRLIBWXI0v+677P8CMQCU7Sg8iu1p0IEsnbx7jymjIhH
QIV7m
Llr+iwo4jYiG96WdH0iUk9yTMwsG6sJdaAA=
-----END CERTIFICATE-----

```

On trouve l'autorité de certification et on récupère son certificat :

```
wget http://crt.harica.gr/HARICA-GEANT-TLS-E1.cer
```

On affiche le certificat avec la commande `openssl x509 -inform der -in HARICA-GEANT-TLS-E1.cer -text` :

```

raphael_laurent@j009-10:~/Téléchargements$ openssl x509 -in
form der -in HARICA-GEANT-TLS-E1.cer -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            42:fd:dc:e1:26:16:07:e1:a5:e6:93:5a:40:01:61:dd
        Signature Algorithm: ecdsa-with-SHA384
        Issuer: C = GR, O = Hellenic Academic and Research

```

```

Institutions CA, CN = HARICA TLS ECC Root CA 2021
    Validity
        Not Before: Jan  3 11:14:21 2025 GMT
        Not After : Dec 31 11:14:20 2039 GMT
    Subject: C = GR, O = Hellenic Academic and Research
Institutions CA, CN = GEANT TLS ECC 1
    Subject Public Key Info:
        Public Key Algorithm: id-ecPublicKey
        Public-Key: (384 bit)
        pub:
            04:03:4f:58:bc:21:d1:96:b6:52:ab:5b:2d:
5e:ff:
            a8:d4:66:ef:cc:ec:b6:d8:12:ba:f8:67:ef:
76:d4:
            1a:dc:44:4e:c5:4e:11:97:8c:51:f3:23:80:
7c:3c:
            13:0a:ec:48:92:3d:a2:3d:02:81:94:05:15:
21:18:
            d4:d2:5f:73:10:3f:34:98:8e:60:95:0a:3e:
c1:5b:
            a4:ea:d8:76:bd:2a:e2:0f:9a:05:9b:17:59:
e8:7b:
            6c:c0:b3:c7:6e:20:fd
        ASN1 OID: secp384r1
        NIST CURVE: P-384
    X509v3 extensions:
        X509v3 Basic Constraints: critical
            CA:TRUE, pathlen:0
        X509v3 Authority Key Identifier:
            C9:1B:53:81:12:FE:04:D5:16:D1:AA:BC:9A:6F:B
7:A0:95:19:6E:CA
        Authority Information Access:
            CA Issuers - URI:http://crt.harica.gr/HARIC
A-TLS-Root-2021-ECC.cer
        X509v3 Certificate Policies:
            Policy: X509v3 Any Policy
        X509v3 Extended Key Usage:
            TLS Web Client Authentication, TLS Web Serv

```

er Authentication

X509v3 CRL Distribution Points:

Full Name:

URI:http://crl.harica.gr/HARICA-TLS-Root-2021-ECC.crl

X509v3 Subject Key Identifier:

E9:99:06:8D:17:1F:AB:FB:96:1A:5A:C8:5B:5E:5D:5E:EC:DA:9C:8F

X509v3 Key Usage: critical

Digital Signature, Certificate Sign, CRL Sign

Signature Algorithm: ecdsa-with-SHA384

Signature Value:

30:66:02:31:00:f6:33:57:1a:69:8d:8e:c2:6b:61:82:60:0d:

51:08:37:2c:12:c8:d3:fd:8b:88:27:71:6b:6c:cd:ab:41:b9:

f7:05:73:6e:31:24:d1:c4:7f:a7:1e:76:39:e8:08:c1:10:02:

31:00:a4:fa:24:e6:06:54:7a:5d:fb:fa:8a:08:a2:68:58:8f:

61:f3:76:1d:f6:dc:60:b5:9b:a2:de:43:67:61:f5:6f:82:55:

25:5f:95:ae:06:bb:ba:ee:74:48:31:bd:85:fe

-----BEGIN CERTIFICATE-----

MIIDNzCCARYgAwIBAgIQQv3c4SYWB+Gl5pNaQAFh3TAKBggqhkJOPQQDAzBsMQsw

CQYDVQQGEwJHUjE3MDUGA1UECgwuSGVsbGVuaWMgQWNhZGVtaWMgYW5kIFJlc2Vh

cmNoIEluc3RpdHV0aW9ucyBDQTEkMCIGA1UEAwwbSEFSSUNBIFRMUyBFQ0MgUm9v

dCBDQSAyMDIxMB4XDTI1MDEwMzExMTQyMVoXDTM5MTIzMTEwMTQyMFowYDELMakG

A1UEBhmCR1IxNzA1BgNVBAoMLkhlbGxlbmljIEFjYWRLbWljIGFuZCBSZXNlYXJj

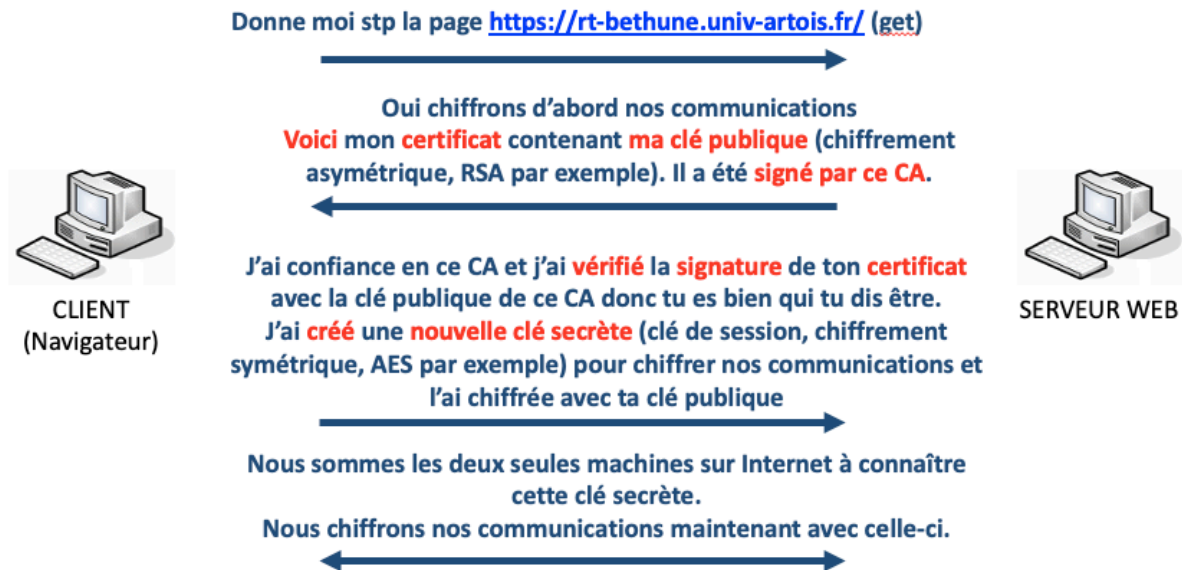
aCBJbnN0aXR1dGlvbG9uMGQ0ExGDAWBgNVBAMMD0dFQU5UIFRMUyBFQ0MgMTB2MBAG

ByqGSM49AgEGBSuBBAAiA2IABANPWLwh0Za2UqtbLV7/qNRm78zsttgSuvh

```
n73bU
GtxETsV0EZeMUfMjgHw8EwrsSJI9oj0CgZQFFSEY1NJfcxA/NJi0YJUKPsF
bp0rY
dr0q4g+aBZsXWeh7bMCzx24g/a0CAS0wggEpMBIGA1UdEwEB/wQIMAYBAf8
CAQAw
HwYDVR0jBBgwFoAUyRtTgRL+BNUW0aq8mm+3oJUZbsowTQYIKwYBBQUHAQE
EQTA/
MD0GCCsGAQUFBzAChjFodHRwOi8vY3J0Lmhhcm1jYS5nci9IQVJJQ0EtVEx
TLVJv
b3QtMjAyMS1FQ0MuY2VyMBEGA1UdIAQKMAgwBgYEVR0gADAdBgNVHSUEFjA
UBggr
BgEFBQcDAgYIKwYBBQUHAwEwQgYDVR0fBDswOTA3oDWgM4YxaHR0cDovL2N
ybC5o
YXJpY2EuZ3IvSEFSSUNBLVRMUy1Sb290LTIwMjEtRUNDLmNybDAdBgNVHQ4
EFgQU
6ZkGjRcfq/uWGl rIW15dXuzanI8wDgYDVR0PAQH/BAQDAgGGMAoGCCqGSM4
9BAMD
A2kAMGYCMQD2M1caaY20wmthgmANUQg3LBLI0/2LiCdx a2zNq0G59wVzbjE
k0cR/
px520egIwRACMQCk+iTmBlR6Xfv6igiiaFiPYfN2HfbcYLWbot5DZ2H1b4J
VJV+V
rga7uu50SDG9hf4=
-----END CERTIFICATE-----
```

HTTPS (simplifié)

La figure ci-dessous résume les grandes lignes du fonctionnement d'HTTPS (sur TLS)



- Écrivez dans votre compte rendu, les réponses aux questions suivantes :
 - Qui donne son certificat ? Pourquoi ?

C'est le serveur web qui donne son certificat pour prouver son identité au client, ensuite il lui envoie sa clé publique pour initier un échange sécurisé en https à l'aide d'un chiffrement asymétrique
 - Comment le certificat est vérifié ? Par qui ?

Le certificat est vérifié par le client qui vérifie que l'Autorité de Certification a bien signé le certificat avec sa clé publique
 - Qui donne la clé de chiffrement de la session ?

C'est le navigateur qui crée et donne sa clé de session
 - La clé de chiffrement de la session est associée à un chiffrement symétrique ou asymétrique ?

Elle est associée à un chiffrement symétrique
 - Comment est-elle transmise ?

Elle est transmise de manière sécurisée en étant chiffrée avec la clé publique du serveur. Ainsi, seul le serveur possède la clé privée correspondante pour la déchiffrer.
 - Au final, client et serveur utilisent un chiffrement asymétrique et symétrique ?

Oui les deux types de chiffrement sont utilisés

- Pourquoi les deux ?
 - Le **chiffrement asymétrique** est utilisé au début pour authentifier le serveur et échanger la clé de session de manière sécurisée sans que personne d'autre ne puisse l'intercepter.
 - Le **chiffrement symétrique** est utilisé ensuite pour le transfert effectif des données car il est beaucoup plus rapide et efficace pour chiffrer de grands volumes de communications.

Demande de signature de certificat (CSR - Certificate Signing Request)

Faites comme si vous étiez le propriétaire du site web **monsiteweb.fr** est que vous vouliez demander un certificat au près d'une autorité de certification (CA).

- Créez avec openssl votre de demande de signature de certificat CSR (*voir memento*)
 - Faites attention à remplir correctement les champs de la demande notamment le **nom de domaine (CN Common Name)** (monsiteweb.fr ici)
 - Par exemple pour rt-bethune on avait :

```
C = FR, ST = Hauts-de-France, O = UNIVERSITE D ARTOIS, CN = rt-bethune.univ-artois.fr
```

- Vérifiez son contenu (*voir memento*)

(Remarque, généralement votre hébergeur de site web vous propose une méthode via une interface web pour faire ceci via le CA Let's Encrypt par exemple)

On crée le certificat et on vérifie son contenu :


```

72:71:
                                ca:c5:d1:dc:02:fc:a9:e9:d4:6d:cd:87:63:
aa:fb:
                                b7:45:8d:2a:ac:fe:43:db:e3:42:00:a1:dd:
c1:75:
                                2a:a0:33:6b:80:5c:60:1e:48:6d:c1:aa:18:
34:91:
                                c9:10:64:0b:2c:23:56:cf:7b:2c:b0:41:89:
01:3d:
                                06:ef:4d:b0:6a:57:f8:85:ea:e2:4a:d3:4a:
92:66:
                                27:0a:e2:64:0c:5a:79:83:c8:e5:ef:d9:d5:
d2:1a:
                                28:2c:fa:35:3c:49:ee:2a:a8:6a:6e:d2:e2:
18:0c:
                                82:cc:cf:0e:70:1f:78:cc:b3:46:44:8f:f8:
98:30:
                                30:1e:2e:bb:bf:4f:28:f1:19:95:15:1a:9e:
54:c8:
                                dd:d5:b9:32:40:86:d3:9f:a2:ea:62:47:67:
4e:0c:
                                72:ad:94:37:e1:56:63:71:dd:9b:b4:95:ac:
de:a5:
                                75:fd:3d:82:99:73:8a:cb:50:31:2b:93:47:
35:78:
                                1d:ce:bb:d5:0f:d2:4c:c8:69:bb:f4:a6:ff:
56:ae:
                                1c:df:64:25:0a:fc:d4:8e:99:4e:5d:60:4b:
e6:dd:
                                e9:29
                                Exponent: 65537 (0x10001)
                                Attributes:
                                    challengePassword      :progtr00
                                Requested Extensions:
                                Signature Algorithm: sha256WithRSAEncryption
                                Signature Value:
                                    a5:6b:74:25:5d:ca:d5:ad:4c:0a:30:5b:c1:68:7d:37:de:
53:

```

```

51:21:39:c6:9e:21:ca:ea:e1:7b:d9:e7:0b:7a:f9:e4:76:
aa:
7a:b0:1e:5f:a5:0b:c3:c8:38:a9:22:68:ba:db:c9:84:49:
58:
0b:37:1d:f6:89:74:56:8e:61:3f:0d:54:93:b1:24:45:0c:
16:
b4:f2:d4:e4:f8:0c:4f:10:f6:a5:33:65:0b:8d:44:85:67:
a1:
11:f5:65:83:e9:35:8b:13:73:ad:d9:1f:29:be:65:ef:76:
2d:
26:25:2c:69:ca:75:9e:aa:73:75:df:27:e2:2b:9d:ad:ee:
38:
d5:04:79:25:1f:e4:14:b9:44:00:4f:50:d9:c9:ed:cc:c9:
c2:
3a:fd:aa:69:ca:06:bf:fc:57:67:f6:77:ca:d9:4c:46:47:
95:
8d:b4:b1:7b:7d:e5:7d:65:fc:43:c1:b1:6f:8d:46:3b:b4:
97:
59:ec:b7:d2:9a:02:12:68:5a:2a:38:e9:05:64:e9:c5:0f:
19:
f4:b4:ee:0d:d6:f5:f8:46:94:01:2e:54:bd:86:06:f3:89:
28:
03:a8:5c:ba:6b:fc:43:f6:92:86:02:35:c3:c6:5a:87:95:
68:
d5:c7:d9:8e:82:c9:be:f2:72:a0:3a:56:42:35:a9:10:eb:
75:
2c:13:46:e4

```

Certicat auto-signé - Self-Signed Certificate

Si vous souhaitez utiliser un certificat uniquement en interne, ou pour tester votre infrastructure, vous pouvez utiliser un **certificat auto-signé**, c'est-à-dire signé par vous-même et donc non signé par une autorité de certification (CA).

- Créez avec openssl un certificat auto-signé pour **mondomaine** (*voir mémento ou ci-dessous - c'est la fin du TP alors on aide plus encore*).

- `openssl req -newkey rsa:2048 -nodes -keyout mondomaine.key -out mondomaine.csr`
(soyez cohérent avec **nom de domaine (CN Common Name)** qui doit valoir mondomaine)
- puis
- `openssl x509 -signkey mondomaine.key -in mondomaine.csr -req -days 365 -out mondomaine.crt`
- Affichez le contenu du certificat.
- `openssl x509 -in mondomaine.crt -noout -text`

Vous pouvez ensuite être votre propre autorité de certification (CA) en créant un certificat CA racine auto-signé, et en l'installant comme certificat de confiance dans votre navigateur local.

- Créez une clé privée (rootCA.key) et un CA racine auto-signé (rootCA.crt)
 - `openssl req -x509 -sha256 -days 730 -newkey rsa:2048 -keyout rootCA.key -out rootCA.crt`
- Créez un fichier texte de configuration `mondomaine.ext` avec le contenu suivant :

```
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
subjectAltName = @alt_names
[alt_names]
DNS.1 = mondomaine
```

Le champ **DNS.1** doit être le domaine de votre site.

- Vous pouvez alors signer votre CSR `mondomaine.csr` avec le certificat `rootCA.crt` de l'autorité de certification racine et sa clé privée :
 - `openssl x509 -req -CA rootCA.crt -CAkey rootCA.key -in mondomaine.csr -out mondomaine.crt -days 365 -CAcreateserial -extfile mondomaine.ext`
- Affichez à nouveau le contenu du certificat `mondomaine.crt`.
 - `openssl x509 -in mondomaine.crt -noout -text`
- Qu'observez-vous maintenant ?

On crée un certificat de domaine `openssl req -newkey rsa:2048 -nodes -keyout mondomaine.key -out mondomaine.csr` :

```

raphael_laurent@j009-10:~$ openssl req -newkey rsa:2048 -nodes -keyout mondomaine.key -out mondomaine.csr
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Hauts-de-France
Locality Name (eg, city) []:Béthune
Organization Name (eg, company) [Internet Widgits Pty Ltd]:UNIVERSITE D ARTOIS
Organizational Unit Name (eg, section) []:UNIVERSITE D ARTOIS
Common Name (e.g. server FQDN or YOUR name) []:mondomaine.fr
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:progtr00
An optional company name []:
raphael_laurent@j009-10:~$

```

On signe le certificat `openssl x509 -signkey mondomaine.key -in mondomaine.csr -req -days 365 -out mondomaine.crt`:

```

raphael_laurent@j009-10:~$ openssl x509 -signkey mondomaine.key -in mondomaine.csr -req -days 365 -out mondomaine.crt
Certificate request self-signature ok
subject=C = FR, ST = Hauts-de-France, L = B\C3\83\C2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ARTOIS, CN = mondomaine.fr

```

On regarde le contenu du certificat :

```

raphael_laurent@j009-10:~$ openssl x509 -in mondomaine.crt -noout -text
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            77:cf:0d:45:4b:57:8d:cc:fd:88:9d:a2:a2:b9:c6:91:e2:95:64:74
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = FR, ST = Hauts-de-France, L = B\C3\83\C2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ARTOIS, CN = mondomaine.fr
        Validity

```

```

        Not Before: May  4 08:47:29 2026 GMT
        Not After  : May  4 08:47:29 2027 GMT
        Subject: C = FR, ST = Hauts-de-France, L = B\C3\83
\C2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ART
OIS, CN = mondomaine.fr
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:e1:d7:7f:db:e3:7e:6f:b0:23:ed:ea:73:
f3:28:
                e3:31:57:f4:de:21:71:4d:74:18:4b:ac:dc:
70:f2:
                d0:3f:d5:cd:da:e0:54:c1:8d:2a:53:07:9c:
59:1c:
                d2:b2:b7:68:e7:75:e5:ac:aa:ea:be:fb:46:
a9:1f:
                96:34:cc:17:9f:bc:e7:35:0b:a8:9e:34:5e:
b2:1f:
                10:92:86:38:47:ce:1c:28:32:f6:83:2b:e6:
2b:fb:
                85:fc:78:31:be:cf:ca:f6:c2:7c:39:17:0e:
59:58:
                07:ec:ce:26:63:ba:2e:4d:c6:29:12:11:28:
39:c0:
                fd:12:03:29:93:2e:f6:97:c3:6b:e4:1e:3e:
85:2a:
                02:f3:ea:9f:aa:c1:a8:73:77:5e:dd:57:bd:
f4:a5:
                d9:f8:4d:64:d5:fe:3f:8d:92:1d:a1:36:59:
9c:70:
                db:6f:d3:49:6d:cc:b2:4a:fa:ab:fe:74:62:
0f:92:
                0e:37:a3:85:90:6f:01:d5:c4:37:bb:69:e5:
6b:52:
                10:cf:09:39:dc:51:a4:d4:82:51:cc:b0:e8:
0e:10:
                b5:cc:22:75:d8:22:4e:10:65:c6:6f:f9:60:

```

44:41:
a1:61:d1:f8:fa:7b:b5:fc:68:de:e3:0b:c2:
a0:b4:
45:b4:c6:d5:d1:05:a8:3a:eb:16:3d:00:47:
07:e8:
9e:c1
Exponent: 65537 (0x10001)
Signature Algorithm: sha256WithRSAEncryption
Signature Value:
df:e9:f3:50:85:fb:96:16:b8:83:ef:72:6b:2a:75:4c:b0:
6a:
50:b3:e5:b9:5f:cf:ce:74:dd:f1:07:80:a3:92:35:d4:b0:
33:
81:4f:70:7d:ca:be:c2:eb:5c:e5:06:e1:5d:c5:f2:cf:3f:
75:
8f:f1:8b:33:5d:2c:bf:57:35:ab:0f:2c:5c:5f:74:8d:b3:
78:
ae:b9:36:16:72:01:51:ba:0e:e4:e1:a6:a6:ad:89:49:88:
9b:
c5:31:fd:c8:c6:e8:93:00:19:1b:9a:30:95:79:c7:cf:1f:
a2:
79:be:8a:7a:4a:2d:6d:2d:92:29:4a:29:a7:20:0a:e6:77:
45:
49:e7:8d:93:7b:6f:23:fe:f5:ee:d7:cc:92:0d:ad:4a:77:
2f:
e4:88:40:8a:13:a8:e6:bb:44:cb:ea:b1:32:b5:05:9f:ae:
3d:
2b:4e:73:99:74:22:d5:86:57:50:1b:7a:3d:fc:69:da:1e:
cc:
a8:d8:9f:99:8c:76:af:57:d2:f7:f1:3c:25:27:ce:1b:55:
f5:
20:cc:d3:2e:a5:8e:23:0f:84:44:bb:55:fc:10:b4:b7:3f:
74:
1b:4b:c9:d7:f9:f5:88:aa:5b:a6:ac:8d:b7:da:3a:d1:d8:
82:
1f:1c:7e:a1:17:8e:14:87:03:0b:fe:4b:fe:ac:90:d2:d6:
6c:
a8:5f:fd:a4

On crée notre certificat root avec la commande `openssl req -x509 -sha256 -days 730 -newkey rsa:2048 -keyout rootCA.key -out rootCA.crt` :

```
raphael_laurent@j009-10:~$ openssl req -x509 -sha256 -days 730 -newkey rsa:2048 -keyout rootCA.key -out rootCA.crt
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:Hauts-de-France
Locality Name (eg, city) []:Béthune
Organization Name (eg, company) [Internet Widgits Pty Ltd]:UNIVERSITE D ARTOIS
Organizational Unit Name (eg, section) []:UNIVERSITE D ARTOIS
Common Name (e.g. server FQDN or YOUR name) []:root.fr
Email Address []:
```

On crée notre fichier mondomaine.ext :

```
echo "authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
subjectAltName = @alt_names
[alt_names]
DNS.1 = mondomaine.fr" > mondomaine.ext
```

On signe notre CSR avec la commande `openssl x509 -req -CA rootCA.crt -CAkey rootCA.key -in mondomaine.csr -out mondomaine.crt -days 365 -CAcreateserial -extfile mondomaine.ext` :

```
raphael_laurent@j009-10:~$ openssl x509 -req -CA rootCA.crt -CAkey rootCA.key -in mondomaine.csr -out mondomaine.crt -days 365 -CAcreateserial -extfile mondomaine.ext
Certificate request self-signature ok
subject=C = FR, ST = Hauts-de-France, L = B\3\83\C2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ARTOIS, CN = mondomaine.fr
Enter pass phrase for rootCA.key:
```

On affiche le contenu du certificat `openssl x509 -in mondomaine.crt -noout -text` :

```
raphael_laurent@j009-10:~$ openssl x509 -in mondomaine.crt  
-noout -text
```

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

39:cb:bf:2d:48:f1:fc:c8:0f:4c:9b:f0:58:fa:b7:a
7:8f:2f:0b:8b

Signature Algorithm: sha256WithRSAEncryption

Issuer: C = FR, ST = Hauts-de-France, L = B\C3\83\C
2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ARTOI
S, CN = root.fr

Validity

Not Before: May 4 08:57:27 2026 GMT

Not After : May 4 08:57:27 2027 GMT

Subject: C = FR, ST = Hauts-de-France, L = B\C3\83
\C2\A9thune, O = UNIVERSITE D ARTOIS, OU = UNIVERSITE D ART
OIS, CN = mondomaine.fr

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit)

Modulus:

00:e1:d7:7f:db:e3:7e:6f:b0:23:ed:ea:73:
f3:28:
e3:31:57:f4:de:21:71:4d:74:18:4b:ac:dc:
70:f2:
d0:3f:d5:cd:da:e0:54:c1:8d:2a:53:07:9c:
59:1c:
d2:b2:b7:68:e7:75:e5:ac:aa:ea:be:fb:46:
a9:1f:
96:34:cc:17:9f:bc:e7:35:0b:a8:9e:34:5e:
b2:1f:
10:92:86:38:47:ce:1c:28:32:f6:83:2b:e6:
2b:fb:
85:fc:78:31:be:cf:ca:f6:c2:7c:39:17:0e:
59:58:
07:ec:ce:26:63:ba:2e:4d:c6:29:12:11:28:
39:c0:

```

85:2a:          fd:12:03:29:93:2e:f6:97:c3:6b:e4:1e:3e:
f4:a5:          02:f3:ea:9f:aa:c1:a8:73:77:5e:dd:57:bd:
9c:70:          d9:f8:4d:64:d5:fe:3f:8d:92:1d:a1:36:59:
0f:92:          db:6f:d3:49:6d:cc:b2:4a:fa:ab:fe:74:62:
6b:52:          0e:37:a3:85:90:6f:01:d5:c4:37:bb:69:e5:
0e:10:          10:cf:09:39:dc:51:a4:d4:82:51:cc:b0:e8:
44:41:          b5:cc:22:75:d8:22:4e:10:65:c6:6f:f9:60:
a0:b4:          a1:61:d1:f8:fa:7b:b5:fc:68:de:e3:0b:c2:
07:e8:          45:b4:c6:d5:d1:05:a8:3a:eb:16:3d:00:47:
                9e:c1
                Exponent: 65537 (0x10001)
X509v3 extensions:
  X509v3 Authority Key Identifier:
    C5:37:7C:60:8F:89:6E:C1:5B:9E:6A:98:C8:E9:7
    1:37:B8:91:FE:ED
  X509v3 Basic Constraints:
    CA:FALSE
  X509v3 Subject Alternative Name:
    DNS:mondomaine.fr
  X509v3 Subject Key Identifier:
    50:A7:81:D2:10:E7:0E:1F:45:E3:25:EF:46:29:2
    1:5E:91:A9:AF:F7
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:
    6b:8f:f1:11:ce:b0:df:b6:1d:20:74:bc:91:ae:00:42:31:
    1d:
    16:33:d9:2a:7a:a6:83:b9:0f:d8:27:b7:43:4b:e2:6c:ec:
    01:
    1f:94:6c:25:0f:a7:8c:35:71:33:6c:ab:a6:28:1b:60:e2:

```

```
3c: 56:49:30:70:b2:f9:40:4d:2e:35:16:7f:ad:b5:65:76:df:
8d: 99:e7:44:a3:f0:74:b5:05:96:12:e7:8f:e4:2e:30:2c:54:
4e: e4:af:6d:37:d1:13:57:63:25:65:d5:f5:de:f2:82:c5:67:
1e: 6c:3d:fb:a9:d5:11:a3:04:5d:0f:51:73:14:5e:46:b4:35:
cc: 95:d9:38:c3:7f:f1:70:c3:d6:af:ee:f6:8b:35:af:02:08:
32: d0:af:5f:67:98:86:c5:30:ad:ca:70:75:13:dd:31:6e:8a:
04: e2:f2:86:f4:29:80:1d:6c:5a:f0:05:5e:0e:46:1f:0f:ba:
1d: 72:ff:09:3b:f5:bb:5f:c3:0e:05:13:63:d9:f8:1e:46:90:
05: bd:6e:42:40:81:28:c1:14:ec:06:a2:9f:f2:5e:da:ac:79:
48: ae:da:fa:e6:75:04:f1:af:d2:89:aa:ce:8c:75:ac:41:6a:
17: ff:22:c6:cd:af:1d:ed:ca:b3:31:d9:3c:d0:78:a7:98:02:
e5: 99:f0:f1:0e
```

On observe un changement d'emmeteur car le certificat est signé par notre autorité root, il y a bien la présence de l'extension X509, le certificat est validé par l'autorité racine de notre domaine.